

컴퓨터공학과

알고리즘분석 (2026년 3학년 1학기)

담당 : 김영오



두원공과대학교

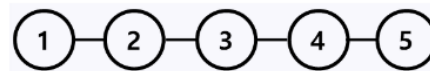
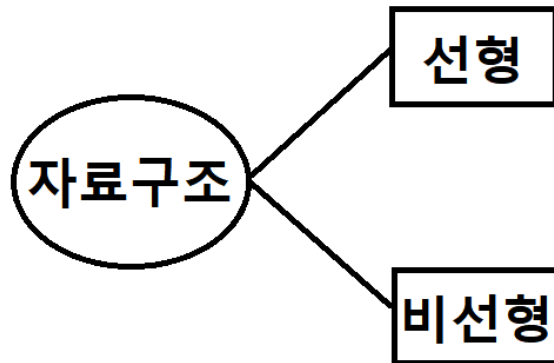


CONTENTS

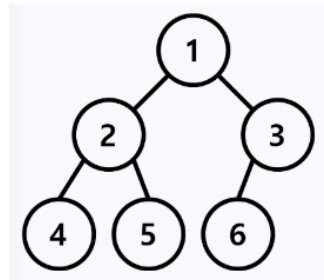
1. 자료구조 분류
2. 기초 알고리즘
3. 실습



선형 자료 vs 비선형 자료



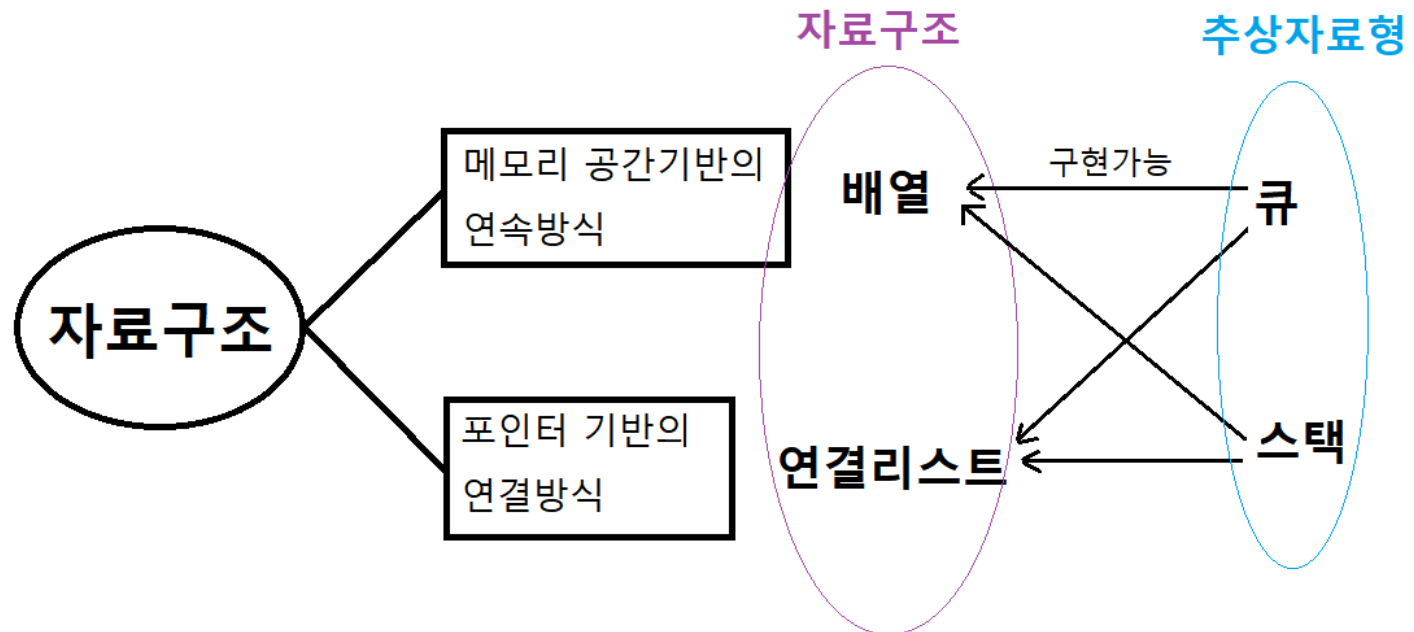
1. 데이터 요소가 순차적으로(Sequential) 배열되는 자료구조
선형 : 자료들 간의 앞뒤 관계가 1:1의 선형관계
2. 단일 레벨로 구성된다.
3. 한 번에 탐색이 가능하고 구현이 쉽다.



1. 데이터 요소가 선형으로 순차적으로 배열되지 않는 자연구조
비선형 : 자료들 간의 앞뒤 관계가 1:n 또는 n:n의 관계
2. 멀티레벨로 구성된다.
3. 탐색이 복잡하고 구현도 어렵지만
메모리를 더 효율적으로 활용할 수 있게 된다.

원시자료형 vs 추상자료형

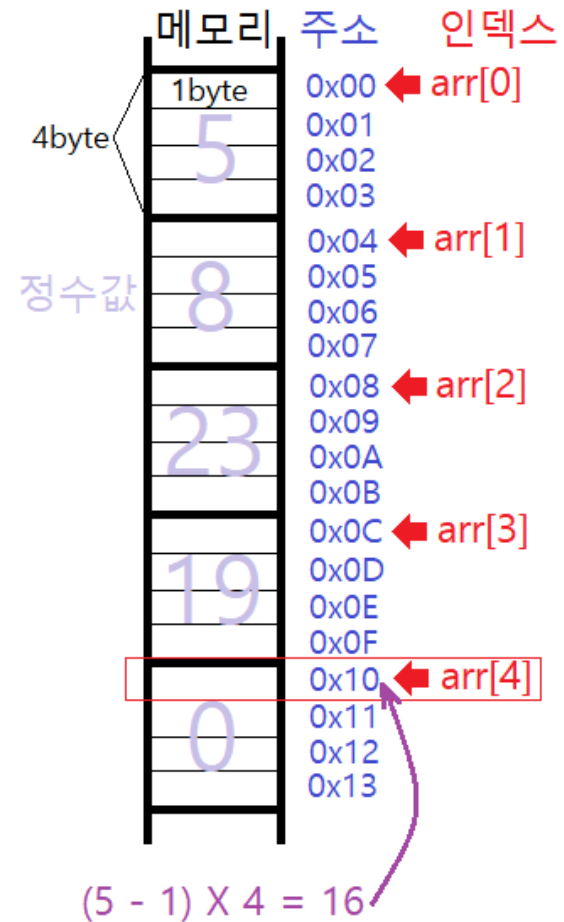
- 원시자료형
 - ✓ 성능에 대한 우선순위가 높은 언어에서 제공하는 자료형
 - ✓ 하드웨어에 좀 더 가까운 원시타입
 - ✓ 원시타입으로 구현했을 때 훨씬 더 빠른 속도로 실행
- 추상자료형
 - ✓ 자료형에 대한 수학적 모델



배열(Array)

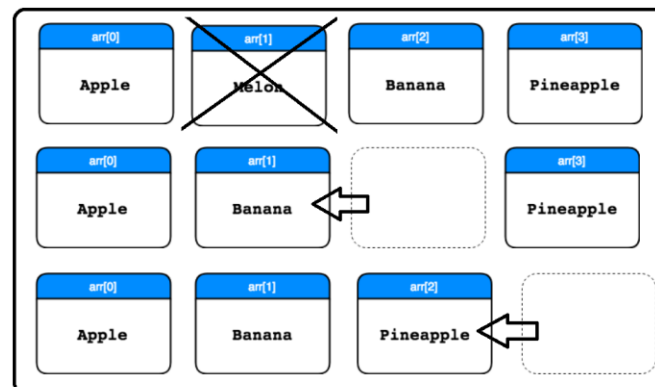
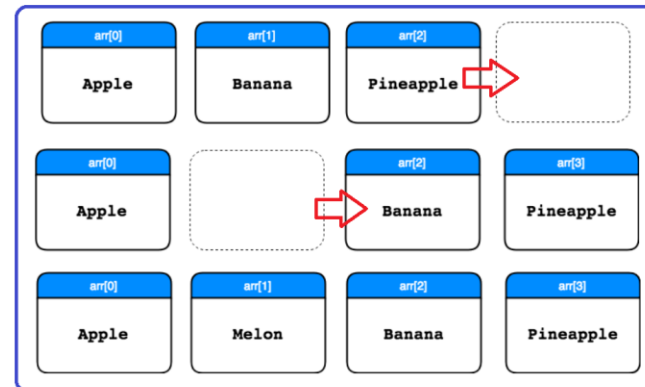
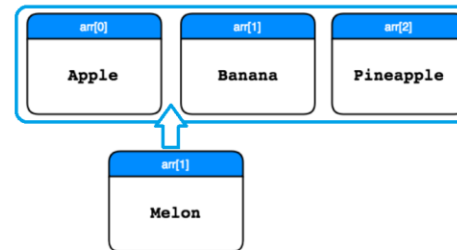
- 여러 데이터를 하나의 이름으로 그룹핑
- 연속된 메모리 공간에 순차적으로 저장
- 원소(element) : 연속되어 배열된 각 값(value) + 인덱스(index)
- 배열의 크기는 고정
- 데이터를 검색하고 확인 : 시간복잡도 $O(1)$ 속도로 작업이 가능
- 저장된 데이터를 삭제, 삽입 : 시간복잡도 $O(n)$
- 용어비교 : Linear List = Ordered List

```
int arr[5] = {5, 8, 23, 19, 0};
```



배열 데이터 삽입, 삭제

- 저장된 데이터를 삭제, 삽입 : 시간복잡도 $O(n)$





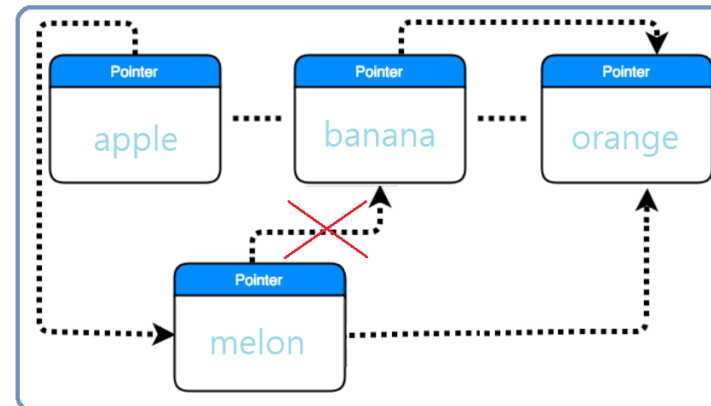
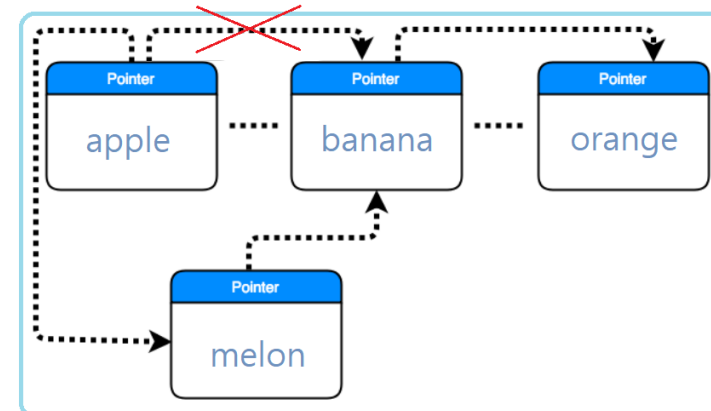
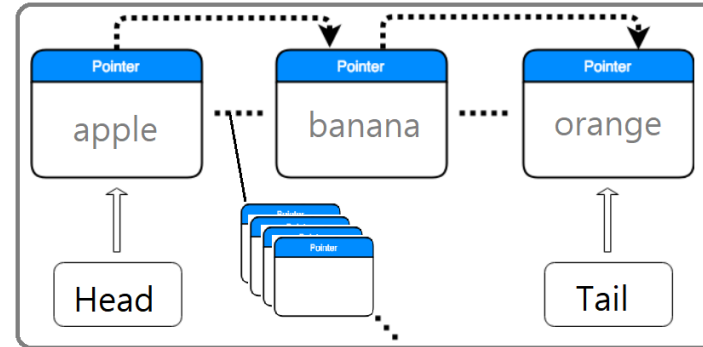
연결 리스트(Linked List)

- 메모리 공간에 불연속적으로 저장
- 각 노드(원소)들은 포인터를 기반으로 연결되어 리스트를 구성
- 첫번째 노드는 헤드(Head), 마지막 노드는 테일(Tail)
- 각 노드는 데이터와 다음 노드를 가리키는 포인터
- n번째 저장된 데이터에 접근 : n-1번까지 순차적으로 접근
- 데이터의 삽입과 삭제가 용이
- 검색 성능이 좋지 않다
- 리스트의 맨 앞 또는 뒤에 데이터를 삽입, 삭제하는 경우 시간복잡도 $O(1)$
- 리스트의 중간에 삽입, 삭제하는 경우 시간복잡도 $O(n)$



연결 리스트 데이터 삽입, 삭제

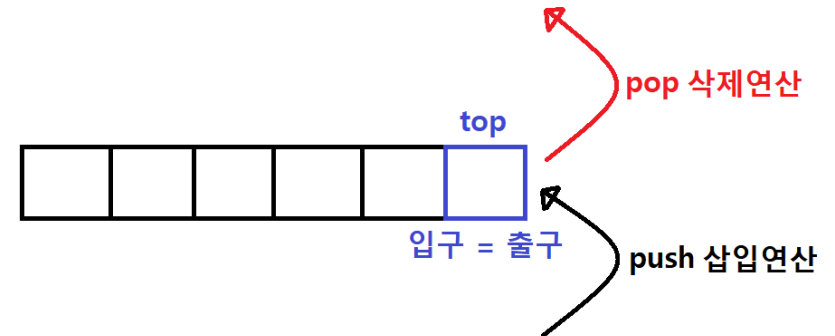
- 리스트의 중간에 삽입, 삭제하는 경우 시간복잡도 $O(n)$





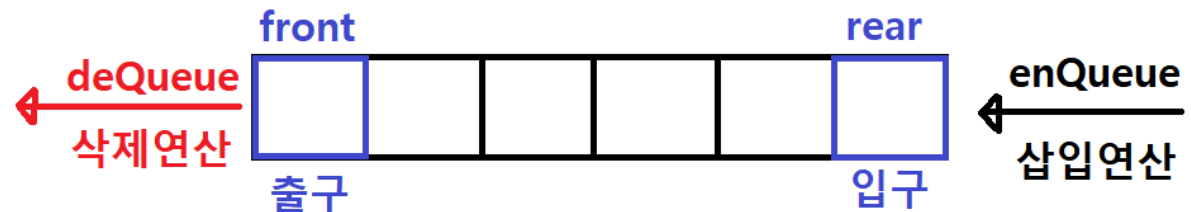
스택(Stack)

- LIFO (Last - In - First - Out 후입선출)
- push() : 요소를 컬렉션에 추가
- pop() : 가장 최근에 삽입된 요소를 제거
- 요소가 없으면 pop : Stack Underflow
- 스택의 크기, 즉 배열의 크기 이상의 자료를 Push : Stack Overflow
- 스택 사용 사례
 - ✓ 웹 브라우저 방문기록 (뒤로 가기) : 가장 나중에 열린 페이지부터 다시 보여준다.
 - ✓ 역순 문자열 만들기 : 가장 나중에 입력된 문자부터 출력한다.
 - ✓ 실행 취소 (undo) : 가장 나중에 실행된 것부터 실행을 취소한다.
 - ✓ 후위 표기법 계산
 - ✓ 수식의 괄호 검사 (연산자 우선순위 표현을 위한 괄호 검사)



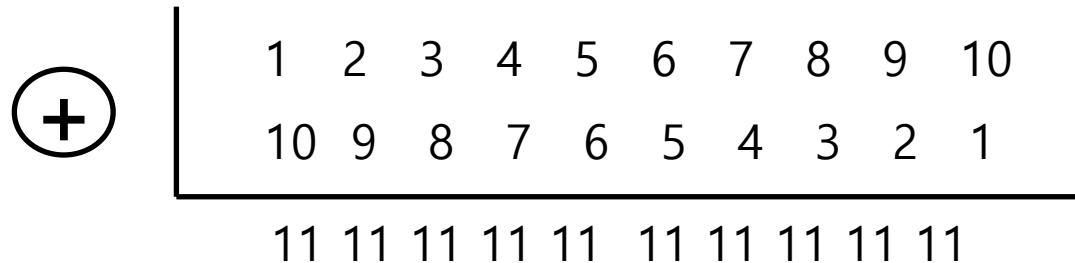
큐(Queue)

- FIFO (First - In - First - Out 선입선출)
- 큐는 한쪽 끝에서 삽입 작업이, 다른 쪽 끝에서 삭제 작업
- 삭제연산만 수행되는 곳: 프론트(front), 삽입연산만 이루어지는 곳 : (rear)
- 삽입연산을 인큐(enQueue)
- 삭제연산을 디큐(dnQueue)
- 큐 사용 사례
 - ✓ 프로세스 관리
 - ✓ 너비 우선 탐색(BFS, Breadth-First Search) 구현
 - ✓ 캐시(Cache) 구현



1 부터 N 까지 더하기

- for
- while
- 내장함수
- 수열의 합 구하는 공식
- range (시작값, 종료값, 증감값)



1	2	3	4	5	6	7	8	9	10
10	9	8	7	6	5	4	3	2	1

11 11 11 11 11 11 11 11 11 11

$$(N + 1) * N / 2$$

■ 최댓값, 최솟값 구하기

- for
- 내장함수
- 숫자 범위 고려
- map (function, iterable)

사용예) `list(map(lambda x: x **3, range(5)))`

